

- [CPSC 375: High-Performance Computing](#)

[CPSC 375: High-Performance Computing](#)

Spring 2026

- [Syllabus](#)
- [Schedule](#)
- [Upload](#)
- [The Cluster Project](#)

Assignment 3

Due 11:59 p.m., February 15, 2026

IMPORTANT! This is an individual assignment. You may discuss broad issues of interpretation, understanding, and general approaches to a solution. However, the development of a specific solution or program code must be your own work. The assignment is expected to be entirely your own, designed and coded by you alone. If you need assistance, please consult your instructor or the TAs. Be sure to read the specific policies outlined in the [Academic Honesty in Computing](#) section.

Problem Definition

Suppose we toss darts randomly at a square dartboard, whose bullseye is at the origin, and whose sides are 2 feet in length. Suppose also that there's a circle inscribed in the square dartboard. The radius of the circle is 1 foot, and its area is π square feet. If the points that are hit by the darts are uniformly distributed (and we always hit the square), then the number of darts that hit inside the circle should approximately satisfy the equation

$$\frac{\text{number in circle}}{\text{total number of tosses}} = \frac{\pi}{4}$$

since the ratio of the area of the circle to the area of the square is $\pi/4$. We can use this formula to estimate the value of π with a random number generator:

```
number_in_circle = 0;
for (toss = 0; toss < number_of_tosses; toss++) {
    x = random double between -1 and 1;
    y = random double between -1 and 1;
    distance_squared = x*x + y*y;
    if (distance_squared <= 1) number_in_circle++;
}
pi_estimate = 4*number_in_circle/((double) number_of_tosses);
```

This is called the Monte Carlo method, since it uses randomness (the dart tosses).

Serial Version. Write a serial program (`monte_serial.c`) that uses a Monte Carlo method to estimate π . Use Linux time utility to measure its running time as follows:

```
#include <time.h>

void main()
{
```

```

time_t start, end;

/* read the input */
...

start = time(NULL);
/* do something */
...

/* done */
end = time(NULL);

printf("Elapsed time = %ld seconds\n", end - start);
}

```

You may want to use `long long ints` for the number of hits in the circle and the number of tosses, since both may have to be very large to get a reasonable estimate of π .

Concurrent Version. Write a concurrent version of your program (`monte_master.c`, `monte_worker.c`) using Linux IPC. You must implement the following:

- The "master" process should spawn M "worker" processes.
- Message queues: The master process sends tasks to a message queue, where each task contains a "chunk" of tosses for the worker processes to independently generate random points and return a partial count.
- Shared memory: A global counter that keeps track of the total number of points landed inside the circle.
- Semaphores: Required to protect all shared memory updates
- Signals: The following signals from the shell should be handled:
 - SIGUSR1: All processes (master+workers) must stop what they are doing. The master process stops enqueueing new tasks, and workers finish current task then block before receiving next.
 - SIGUSR2: All processed may resume their work.
 - SIGINT: All processes must terminate. Master broadcasts signal to workers. All processes cleanup and exit.

Programming Notes

- Compile with:

```

$ gcc -Wall -o monte_master monte_master.c
$ gcc -Wall -o monte_worker monte_worker.c

```

- Run with:

```

$ ./monte_master -M 2 -N 5000000 -C 250000 -S 1

```

where M is the number of worker processes, N the number of sample points, C the number of chunks, and S the seed for the random number generator.

Handin

When completed, submit your C source `monte_master.c`, `monte_worker.c` on the course website.

- **Welcome: Sean**

- [Logout](#)

